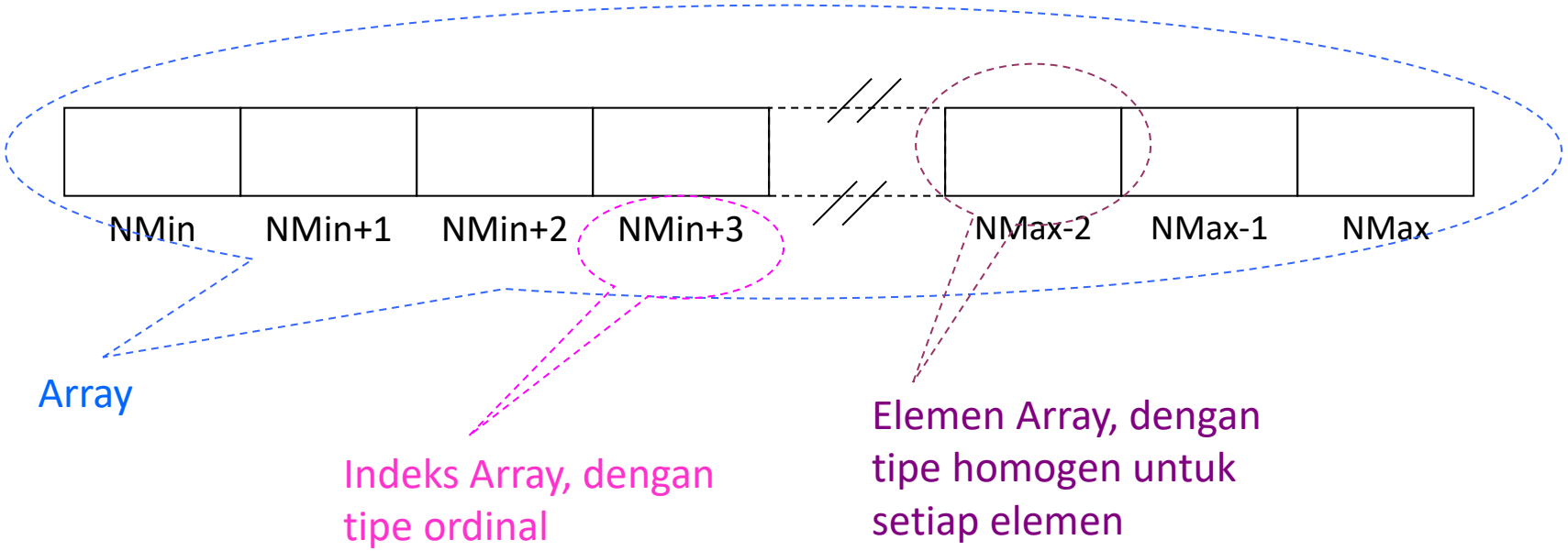
IF1210 Algoritma dan Pemrograman 1

Skema Standar (Bag. 3): Skema Pemrosesan Sekuensial pada Array

Tim Pengajar IF1210

Sekolah Teknik Elektro dan Informatika

Array



- **Array** mendefinisikan **sekumpulan** (satu atau lebih) elemen **bertype sama**
- Setiap elemen tersusun secara terurut (kontigu) dan dapat diakses dengan menggunakan **indeks**

Deklarasi Array sbg. **Variabel** dalam Notasi Algoritmik (1)

- Deklarasi array sebagai variabel di **KAMUS**

  nama-var : array [*idmin*..*idmax*] of *type-elmt*

  - Deklarasi variabel array dengan nama *nama-var* dengan indeks elemen terkecil *idmin* dan indeks terbesar *idmax*
    - Indeks bertype ordinal, misalnya integer. Indeks bisa dimulai dari nilai berapa pun (ini berbeda dengan beberapa bahasa pemrograman, misalnya Python)
  - Type elemen array ditentukan oleh *type-elmt*

Deklarasi Array sbg. **Variabel** dalam Notasi Algoritmik (2)

- Cara akses sebuah elemen:

$$\texttt{nama-var}_{\texttt{indeks}}$$

- Contoh deklarasi array

```
Tab : array[1..100] of integer
```

- Contoh akses elemen:

$\underline{\texttt{output}}(\texttt{Tab}_5)$

$\texttt{x} \leftarrow \texttt{Tab}_1 + \texttt{Tab}_6$

$\texttt{Tab}_9 \leftarrow 9$

Untuk memudahkan dalam mengetik algoritma (untuk jawaban yang diketik), akses sebuah elemen array dapat dituliskan menggunakan sintaks:

nama-var[indeks]

Misalnya: Tab_1 dapat ditulis Tab[1]

Deklarasi Array sbg. **Type** dalam Notasi Algoritmik

- Array dapat menjadi salah satu komponen dalam type bentukan.
- Contoh: deklarasi type **TabInt**:

```
KAMUS
    constant NMax : integer = 100
    type TabInt : array [1..NMax] of integer
    { Variabel }
    T : TabInt
ALGORITMA
    T1 ← 0   { contoh cara akses }
```

  - TabInt adalah type dengan komponen tunggal, yaitu sebuah array of integer dengan indeks dari 1 s.d. Nmax (konstanta)
  - T adalah variabel bertype TabInt

Contoh Lain

```
KAMUS
  TabKata : array [1..100] of string
  TabJumlahHari : array [1..12] of integer
  type Point : < x : integer,
                 y : integer>
  TabTitikSurvey : array [1..10] of Point
```

- **Domain**:
  - Domain array sesuai dengan pendefinisian indeks
  - Domain isi array sesuai dengan jenis array
- Cara mengacu sebuah elemen: melalui indeks

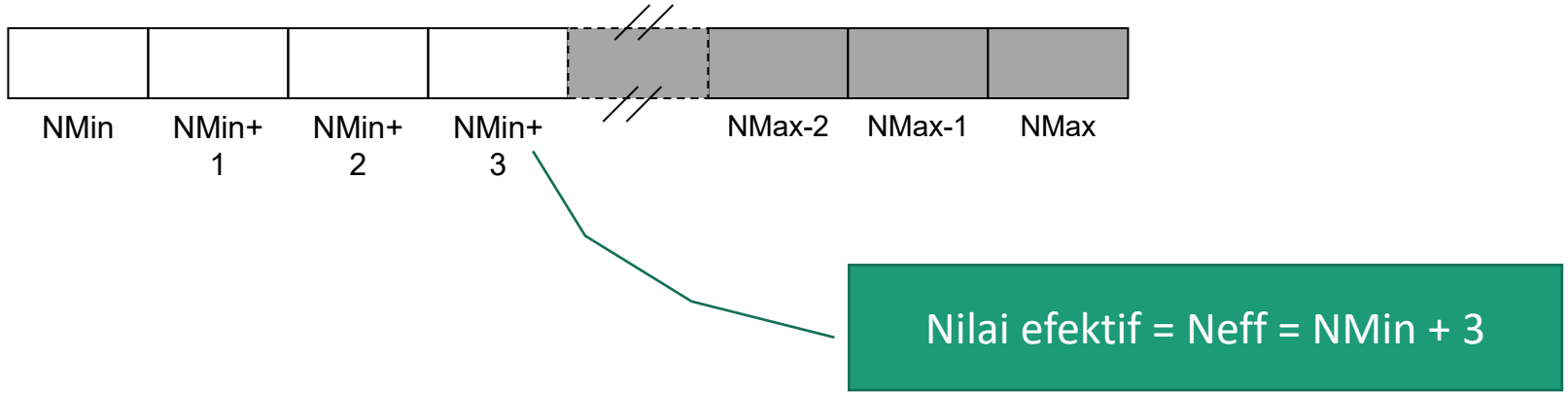
  $TabKata_i$ {jika i terdefinisi}

  $TabKata_7$

  $TabJumlahHari_3$

  $TabTitikSurvey_4.x$ {akses komponen x dari Point pada array elemen ke-4}

Array Rata Kiri Eksplisit (1)



- Pada pembahasan berikutnya, kita mendefinisikan array yang hanya terisi sebagian secara **"rata kiri",** yaitu terisi/terdefinisi dari elemen ke-NMin dan dan **"eksplisit"**, yaitu elemen terakhir yang terdefinisi melalui nilai efektif (Neff) array
  - Pada contoh di Elemen NMin+4 s.d. NMax dianggap tidak terdefinisi (oleh karena itu, tidak boleh diakses)

Array Rata Kiri Eksplisit (2)

- Deklarasi array dan nilai efektif secara terpisah, contoh:

```
KAMUS

constant NMax : integer = 100
type TabInt : array [1..NMax] of integer

{ Jika diperlukan sebuah tabel, maka akan dibuat deklarasi sbb. }
N : integer    { indeks efektif maksimum tabel yang terdefinisi,
                 0 ≤ N ≤ NMax }
T : TabInt     { tabel integer }
```

- Selalu harus didefinisikan 2 buah variabel (N, T) untuk setiap array

Array Rata Kiri Eksplisit (3)

- Tabel dan nilai efektif dikumpulkan dalam satu struktur type, contoh:

```
KAMUS
  constant NMax : integer = 100
  type TabInt : < Tab : array [1..NMax] of integer,
                  Neff : integer { indeks efektif tabel yang
                                   terdefinisi, 0 ≤ Neff ≤ NMax } >

{ jika diperlukan sebuah tabel, maka akan dibuat deklarasi
  sebagai berikut: }
  T1 : TabInt    { Cara akses elemen:
                   T1.Tab_i untuk akses elemen ke-i dari T1.Tab
                   T1.Neff untuk akses nilai efektif - Neff }
```

Array dengan Elemen Type Bentukan

- Elemen array dapat bertype dasar maupun bentukan
  - Contoh: array integer, real, Point, dll.
- Contoh deklarasi array of Point:

```
KAMUS
  constant NMax : integer = 100
  type Point : < x : integer; y : integer > { Titik di bidang kartesius }
  type TabPoint : < Tab : array [1..NMax] of Point;
                    Neff : integer { indeks efektif tabel yang
                                     terdefinisi, 0 ≤ Neff ≤ NMax } >
  { Contoh deklarasi: }
  TP : TabPoint
       { Cara akses elemen:
         TP.Tab_i.x untuk akses bagian x dari elemen ke-i dari TP.Tab
         TP.Tab_i.y untuk akses bagian y dari elemen ke-i dari TP.Tab
         TP.Neff untuk akses nilai efektif - Neff }
```

Pemrosesan Sekuensial pada Array

Pemrosesan Sekuensial pada Array

- Merupakan **pemrosesan sekuensial tanpa mark**
- Dimungkinkan adanya akses langsung jika indeks terdefinisi
  - First-Elmt adalah elemen tabel dengan indeks terkecil
  - Next-Elmt dicapai melalui suksesor indeks
- Model akses sekuensial tanpa mark
  - kondisi berhenti adalah jika indeks sudah mencapai harga indeks yang terbesar yang telah terdefinisi
- Tabel tidak mungkin “kosong”
  - jika kita mendefinisikan tabel, maka minimal mengandung sebuah elemen

Skema Pemrosesan Sekuensial

```
KAMUS UMUM PEMROSESAN ARRAY
constant NMin : integer = 1     { batas bawah }
constant NMax : integer =  100 { batas atas }
type
    ElType : ... { suatu type terdefinisi, misalnya integer }
{ Variabel }
    i : integer[NMin..NMax]
    T : array [NMin..NMax] of ElType  { array berelemen ElType }
{ Deklarasi Prosedur }
    procedure Inisialisasi { persiapan sebelum pemrosesan }
    procedure Proses (input X : ElType) {proses current-elmt array T}
    procedure Terminasi { penutupan setelah  pemrosesan selesai }
```

```
{ SKEMA PEMROSESAN ARRAY T untuk indeks [NMin..NMax] }
{ Traversal Array T untuk indeks bernilai NMin..NMax }
```

```
{ Skema }
    Inisialisasi
    i traversal[Nmin..Nmax]
        Proses(Ti)
    Terminasi
```

Skema Pemrosesan Sekuensial

```
KAMUS UMUM PEMROSESAN ARRAY
constant NMin : integer = 1     { batas bawah }
constant NMax : integer =  100 { batas atas }
type
    ElType : ... { suatu type terdefinisi, misalnya integer }
{ Variabel }
    i : integer[NMin..NMax]
    T : array [NMin..NMax] of ElType  { array berelemen ElType }
{ Deklarasi Prosedur }
    procedure Inisialisasi { persiapan sebelum pemrosesan }
    procedure Proses (input X : ElType) {proses current-elmt array T}
    procedure Terminasi { penutupan setelah  pemrosesan selesai }
```

```
{ SKEMA PEMROSESAN ARRAY T untuk indeks [NMin..NMax] }
{ Traversal Array T untuk indeks bernilai NMin..NMax }
```

```
{ Skema }
    Inisialisasi
    i traversal[Nmin..Nmax]
        Proses(Ti)
    Terminasi
```

Diasumsikan semua elemen array T sudah terisi (terdefinisi)

Skema Pengisian dan Penulisan Isi Array

(Skema Traversal terhadap Array)

Menuliskan Isi Tabel Secara Mundur

```
Program TULISTABELMundur
{Menuliskan isi tabel dari indeks terbesar ke indeks terkecil}

KAMUS
constant NMin : integer = 1
constant NMax : integer = 100

T : array [NMin..NMax] of integer
i : integer[NMin..NMax]

ALGORITMA

   { Di titik ini: TabelInt[NMin..NMax] sudah diisi,
      algoritma berikut hanya menuliskan mundur }
   i traversal [NMax..NMin]
        output (Ti)
```

Mengisi Tabel, Jumlah Elemen Diketahui

```
Program ISITABEL1
{ Traversal untuk mengisi, dengan membaca nilai setiap elemen tabel
dari keyboard  jika banyaknya elemen tabel yaitu N diketahui. Nilai
yang dibaca akan disimpan di T_NMin s.d. T_N. Nilai N harus dalam daerah
nilai indeks yang valid. }
Kamus
   constant NMin: integer = 1    { NMin : batas bawah indeks }
   constant NMax: integer = 100 { NMax : batas atas indeks }

   i : integer[NMin..NMax]
   T : array [NMin..NMax] of integer
   N : integer
Algoritma
     { Inisialisasi }
     repeat
          input (N)
     until (N >= NMin) and (N <= NMax);
     { Pengisian array dari pembacaan dari keyboard }
     i traversal [Nmin..N]
          input (T_i)
```

Mengisi Tabel, Jumlah Elemen Diketahui

```
Program ISITABEL1
{ Traversal untuk mengisi, dengan membaca nilai setiap elemen tabel
dari keyboard  jika banyaknya elemen tabel yaitu N diketahui. Nilai
yang dibaca akan disimpan di T_NMin s.d. T_N. Nilai N harus dalam daerah
nilai indeks yang valid. }
Kamus
   constant NMin: integer = 1    { NMin : batas bawah indeks }
   constant NMax: integer = 100  { NMax : batas atas indeks }

   i : integer[NMin..NMax]
   T : array [NMin..NMax] of integer
   N : integer
Algoritma
     { Inisialisasi }
     repeat
          input (N)
     until (N >= NMin) and (N <= NMax);
     { Pengisian array dari pembacaan dari keyboard }
     i traversal [Nmin..N]
          input (T_i)
```

Catatan: N sebenarnya adalah indeks maksimum efektif. Tetapi, karena NMin=1, maka N juga merupakan jumlah elemen tabel yang terdefinisi

Mengisi Tabel, Jumlah Elemen Tidak Diketahui (1)

```
Program ISITABEL2
{ Traversal untuk mengisi, dengan membaca nilai setiap elemen
tabel dari keyboard  yang diakhiri dengan 9999. Nilai yang
dibaca akan disimpan di T_NMin s/d T_N, nilai N harus berada dalam
daerah nilai indeks yang valid, atau 0 jika tabel kosong. }

KAMUS
constant NMin : integer = 1   { NMin : batas bawah indeks }
constant NMax : integer = 100 { NMax : batas atas indeks }

i : integer[NMin..NMax]
T : array [NMin..NMax] of integer
N : integer { indeks efektif tabel, 0 jika tabel kosong }
x : integer { nilai yg dibaca & akan disimpan sbg elemen
              tabel }

ALGORITMA
    ... { next slide }
```

Mengisi Tabel, Jumlah Elemen Tidak Diketahui (2)

```
Algoritma
   i ← NMin            { Inisialisasi }
   input (x)           { First element }

   while (x ≠ 9999) and (i <= NMax) do
       Ti ← x          { Proses }
       i ← i + 1
       input (x)       { Next element }
   { x = 9999 or i > NMax }

   if (i > NMax) then
       output ("Tabel sudah penuh")
   N ← i - 1
```

Mengisi Tabel, Jumlah Elemen Tidak Diketahui (2)

```
Algoritma
   i ← NMin            { Inisialisasi }
   input (x)           { First element }

   while (x ≠ 9999) and (i <= NMax) do
       Ti ← x          { Proses }
       i ← i + 1
       input (x)       { Next element }
   { x = 9999 or i > NMax }

   if (i > NMax) then
       output ("Tabel sudah penuh")
   N ← i - 1
```

Jika pada saat read pertama kali sudah diisi "9999", maka N akan berisi NMin – 1. Karena NMin =1, proses ini aman (N diisi 0). Hati-hati jika NMin ≠ 1.